

Exploration of Rewards Shaping and Immitation Learning in Chess

26.01.2026

Jakob Lambert-Hartmann

Contents

Introduction	2
Methods	2
Architecture	2
Behavior Cloning	3
Puzzle Data	4
Grandmaster Data	5
Rewards Shaping	6
Evaluation	6
Results	6
Furter Work	8
Bibliography	9
Appendix	10

Introduction

Reinforcement learning (RL) is a field of machine learning concerned with decision-making. It is used in scenarios where an intelligent agent (e.g. chess player) takes actions in an environment (e.g. chess game) in order to maximize a reward (e.g. win). The behavior of an agent is called a policy. If rewards are sparse, it is difficult for an agent to learn a good policy. Winning a game of chess for instance is a sparse reward. It is difficult for an agent to learn chess with win as only reward, as it is very unlikely for an agent to randomly win a game. To help agents learn policies faster reward shaping introduces further rewards that align with the original goal and help to nudge the model in the right direction. Reward shaping however introduces a new set of problems. If the newly introduced rewards are not aligning with the ultimate goal for instance, the model may choose to prioritize those and ignore the real goal (reward hacking). [1]

Imitation learning is a paradigm in reinforcement learning, in which an agent learns a policy through a set of expert demonstrations, rather than an explicit reward function. Behavior cloning is a specific imitation learning technique, which utilizes supervised learning to learn this policy. However, expert demonstrations used for behavior cloning are often not uniformly sampled from the state space leading to poor performance. [2]

One Idea for mitigating the drawbacks of both reward based reinforcement learning and behavior cloning would be to combine the two approaches. Behavior cloning as a pretraining could reduce the need for reward shaping, while finetuning with reward based reinforcement learning could iron out the sampling bias from the expert demonstrations. In this document we investigate the interaction between pretraining with behavior cloning and finetuning with reward shaping using different sets of expert demonstrations and reward functions. We will apply this to the domain of chess, as available data is well split into different sampling biases.

Methods

In this chapter we will introduce the hyperparameters that will be compared in findings. We will first introduce the different neural network architectures. Then, we will introduce the datasets used for behavior cloning and the reward sets used for reinforcement learning. Finally, we will discuss the method used to compare the different models

Architecture

We trained models with three different architectures: a fully connected network with 3 hidden layers with 512 neurons each (see Figure 7), a convolutional neural network (see Figure 5), and a residual block network (see Figure 6).

While the hidden layers of the agent differ, they share the same input and output representation. The network **input** contains the current position, which consists of a piece encoding (12 pieces: pawn, rook, knight, bishop, queen, king for both black and white) for each square on the chess board (8×8), resulting in an input tensor $b \in \mathbb{R}^{8 \times 8 \times 12}$.

The **output** consists of two tensors representing from which square to move m_{from} to which square m_{to} . A joint probability is created by computing from-to logits $M = m_{\text{from}} m_{\text{to}}^T$, masking out illegal moves (see Figure 1), and computing the softmax $\pi(b) = \sigma(\text{mask}(m_{\text{from}} m_{\text{to}}^T))$.

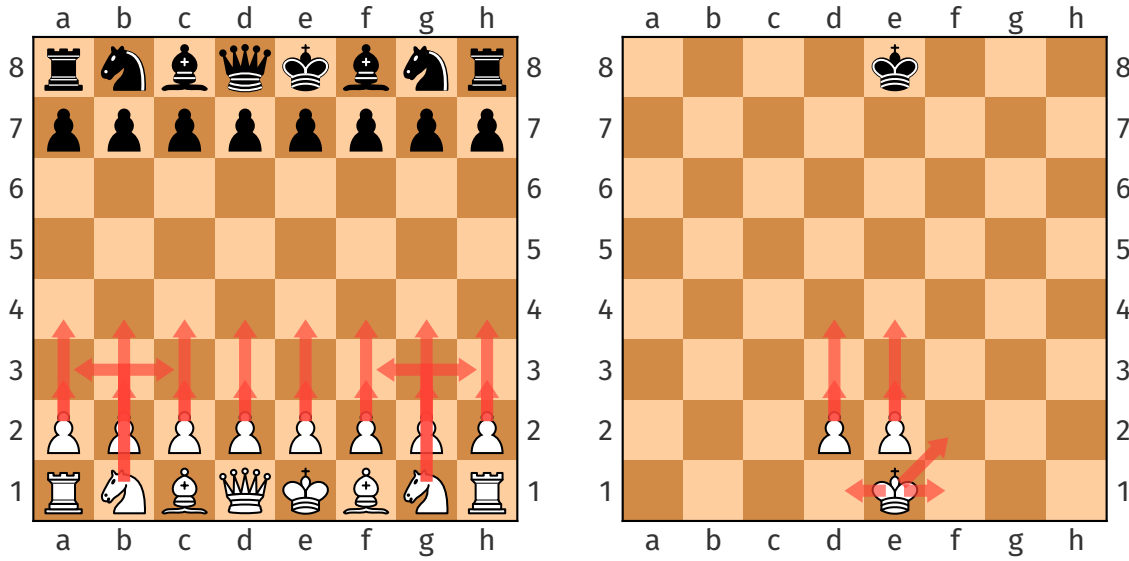


Figure 1: Red arrows show all the legal moves in both positions. As the number of legal moves changes depending on position, the model masks out illegal moves for to generate actions.

Behavior Cloning

For an agent to be successful in Chess it needs to master strategies and tactics. **Tactics** defines moves that achieve short term gains. This may be capturing opponent pieces or checkmating the opponent king. **Strategy** on the other hand defines moves with a long term benefit. This could be positioning a piece to put pressure on the opponents king, which may lead to a checkmate in 20-30 moves. See Figure 2 for example positions.

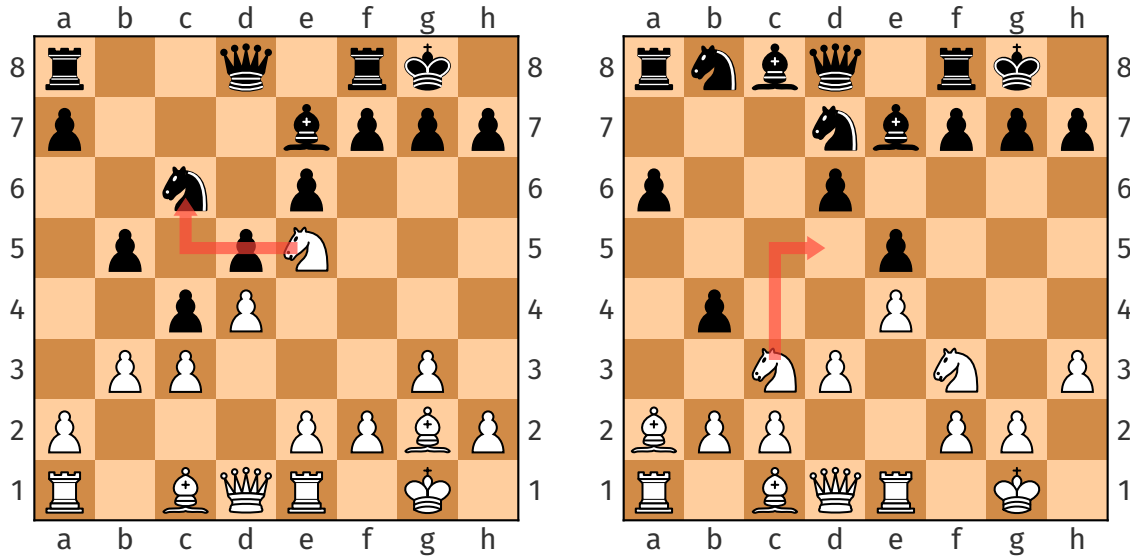


Figure 2: The position on the left has a tactical motif, where the black knight can be captured by white, as it is not defended. On the other hand the right position has a strategic motif, where the knight is attacked and has to retreat. There are multiple available squares, but d5 is the best option. There is no immediate gain, however in the long run the knight will be able to exert more pressure on the opponent from this square.

We chose two datasets, one biased towards strategic, and the other towards tactical positions. For a fair comparison, the datasets were reduced to 6 million positions. For both dataset we trained one model on one epoch, and one model was the best of 10 epochs. In the following sections we discuss both datasets and their respective bias.

Puzzle Data

Players that want to improve their tactical perception can train on puzzles. A chess puzzle contains a position with a tactical motif (e.g. checkmate,...). A player then needs to play 4-5 moves to prove their understanding of the motif.

There are many published datasets containing chess puzzles, such as the Lichess puzzle dataset [3]. Positions in these datasets are highly biased in that they solely contain tactical positions, underrepresenting strategic motifs. Furthermore, positions in such datasets always contain a tactical motif. The lack of negatives may lead the model to be tactically overconfident, leading to false positives (see Figure 3).

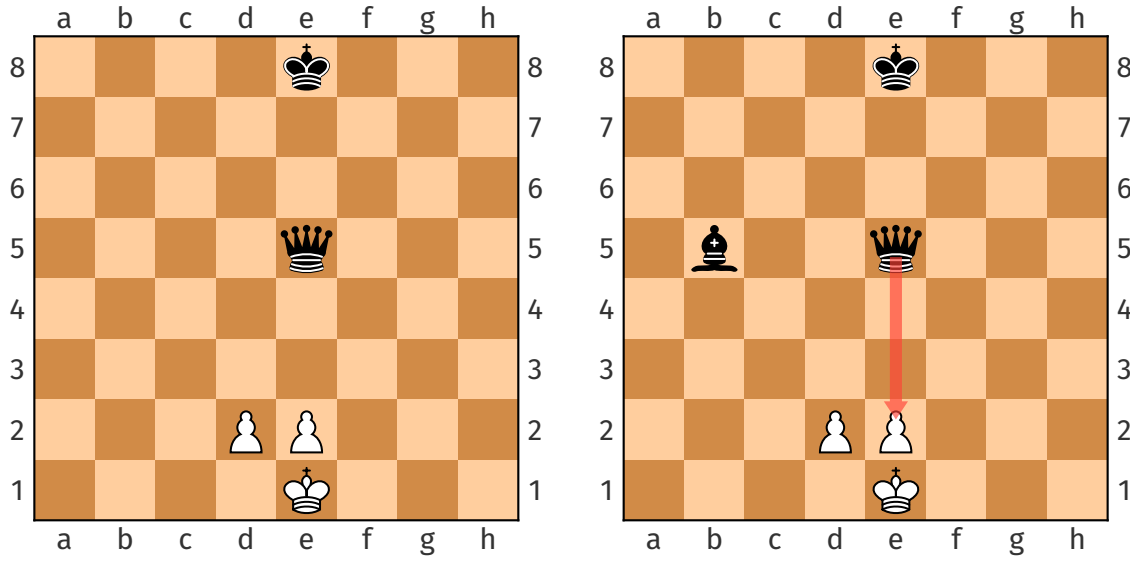


Figure 3: both positions are similar except for the black bishop on d5. In the position with the Bishop (right) black can win the game in one move by checkmating with the queen on e2. playing queen e2 in the other position will lose the game.

Grandmaster Data

Grandmaster games on the other hand are less biased than the puzzle dataset. They cover entire games, and therefore contain positions with tactical motives, and strategic motives. However, they still are biased in that they only represent top level play. Furthermore, grandmasters are able to recognize a losing position earlier than club level players. As such, they often resign multiple moves before an unavoidable checkmate (see [4]). This underrepresentation of checkmates, may make it difficult for our model to convert a winning position into a checkmate (see Figure 4).

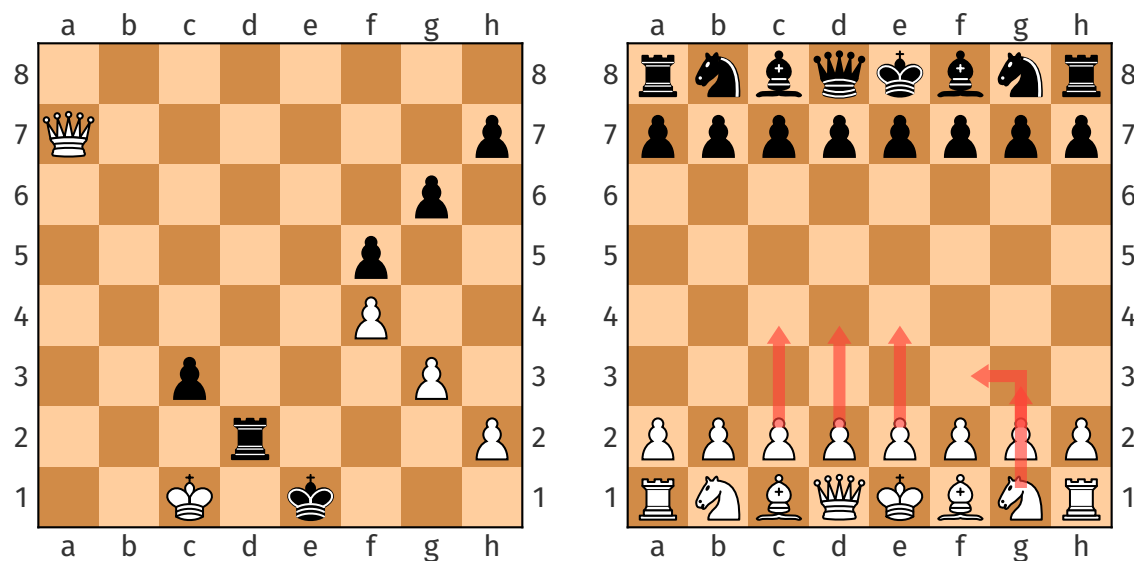


Figure 4: The position on the left shows the last position of a game by the two former world champions Garry Kasparov and Veselin Topalov. Black resigns in this completely lost position without waiting for a checkmate. The position on the right shows different first moves played by masters, showing that there is more than one viable move per position.

Rewards Shaping

Evaluation

To evaluate the model performance, we let the models play against each other. Each model played 500 games as white and 500 games as black against each other model with the same architecture (i.e. cnn models only played cns, etc.).

In chess a win yields one point, a loss zero, and a draw half a point. To compare the models we summed points for each matchup (two models playing 500 games). We normalized the point values to the range $[0, 1]$, which represents the portion of available points the model received.

Results

The models trained with supervised training performed much better on the puzzle test set. However a good accuracy in the masters game is difficult, as a position may have multiple viable moves, and training datapoints.

	dataset	epochs	linear	cnn	resnet
	puzzle	1	62.0%	69.9%	74.6%
		10	65.8%	73.3%	81.7%
	grand masters	1	31.0%	38.5%	41.5%
		10	31.8%	39.6%	43.7%

	architecture	cnn	resnet
train	rewards		
gm_1	none	0.396	0.797
	r_0	0.605	0.411
	r_1	0.650	0.409
	r_2	0.572	0.424
gm_10	none	0.434	0.870
	r_0	0.714	0.387
	r_1	0.694	0.398
	r_2	0.654	0.399
pz_1	none	0.400	0.830
	r_0	0.478	0.410
	r_1	0.564	0.410
	r_2	0.433	0.409
pz_10	none	0.446	0.925
	r_0	0.672	0.403
	r_1	0.515	0.403
	r_2	0.520	0.403
untrained	none	nan	nan
	r_0	0.266	0.408
	r_1	0.235	0.407
	r_2	0.251	0.399

	architecture	cnn	resnet
train			
gm_1		0.556	0.510
gm_10		0.624	0.513
pz_1		0.469	0.515
pz_10		0.538	0.533
untrained		0.251	0.405

architecture	cnn	resnet
rewards		
none	0.419	0.855
r_0	0.547	0.404
r_1	0.532	0.405
r_2	0.486	0.407

Furter Work

Bibliography

- [1] Tim Miller, “Reward Shaping.” [Online]. Available: <https://gibberblot.github.io/rl-notes/single-agent/reward-shaping.html>
 - [2] “Imitation Learning, Stanford University.” [Online]. Available: https://web.stanford.edu/class/cs237b/pdfs/lecture/lecture_10111213.pdf
 - [3] “Lichess chess puzzle dataset.” [Online]. Available: <https://www.kaggle.com/datasets/tianmin/lichess-chess-puzzle-dataset>
 - [4] “All GM Chess Games on Chess.com.” [Online]. Available: <https://www.kaggle.com/datasets/dimitrioskourtikakis/gm-games-chesscom>
-

Appendix

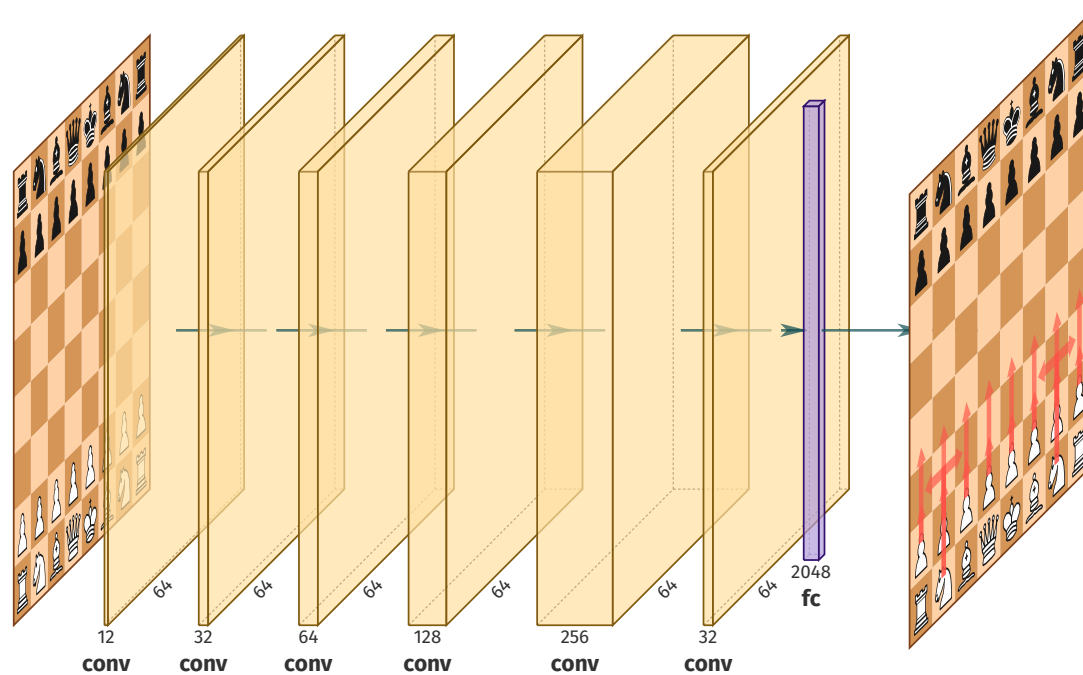


Figure 5: Architecture of the convolutional model used.

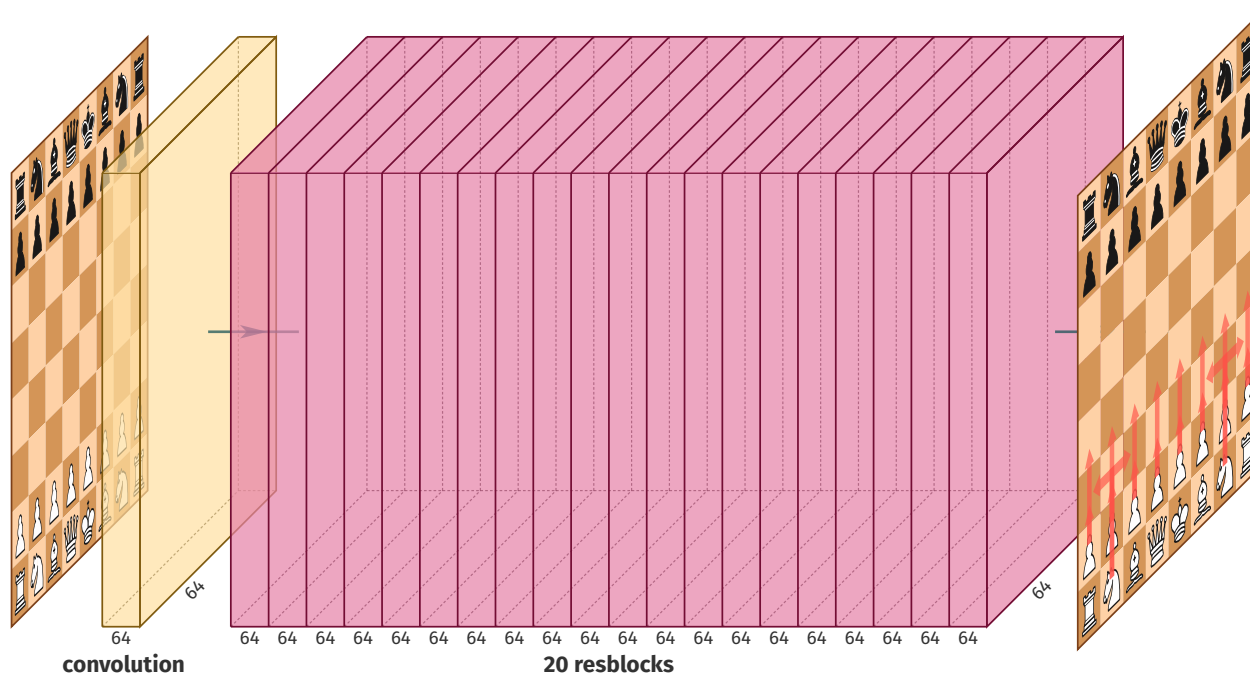


Figure 6: Architecture of the residual block model used.

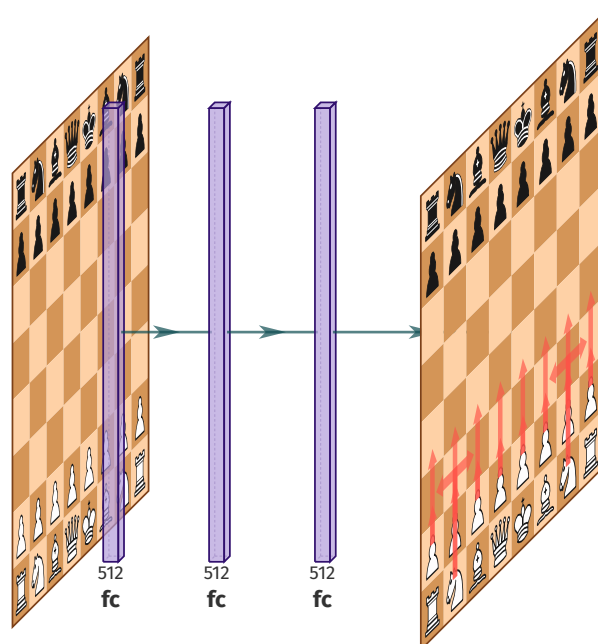


Figure 7: Architecture of the linear model used.